

Aurora AI - Interview FAQs

Q1: Can you explain the overall architecture of Aurora AI?

Aurora AI is built using a Microservices Architecture. It consists of four isolated FastAPI services running concurrently: the Orchestrator Agent (Port 10000) acts as the API gateway and natural language interface; the Ranking Engine (Port 8001) handles FAISS vector similarity; the Event Processor (Port 8002) manages real-time user telemetry and in-memory feature storage; and the RAG/Knowledge Graph Service (Port 8003) generates contextual explanations. They communicate via internal HTTP requests.

Q2: Why did you choose FastAPI over Django or Flask?

FastAPI was chosen for its high performance (powered by Starlette and Pydantic) and native async support. Since our architecture relies on multiple microservices communicating with each other, non-blocking asynchronous I/O is critical. Additionally, FastAPI's automatic data validation via Pydantic ensures robust data contracts between the microservices.

Q3: How did you manage the strict 512MB RAM constraint on Render?

Memory optimization was one of the biggest challenges. Initially, the RAG service used heavy ML libraries like PyTorch and Transformers, which caused Out-Of-Memory (OOM) crashes. I resolved this by ripping out the heavy neural networks and building a lightweight 'IntelligentMetadataExtractor'. This engine uses deterministic heuristics and keyword parsing against our movies dataset, combined with NetworkX graph traversals, to generate high-quality AI insights with a fraction of the memory footprint.

Q4: How does the recommendation system work under the hood?

The recommendation engine uses a hybrid approach. First, we use FAISS (Facebook AI Similarity Search) running on CPU to quickly retrieve nearest neighbors based on pre-computed embeddings. Second, we utilize a Knowledge Graph built with NetworkX to find the shortest path between user preferences and movie entities. Finally, the Event Processor updates user preference weights in real-time based on their click events, dynamically adjusting the ranking scores.

Q5: Why did you use Vanilla JavaScript instead of React or Vue?

To ensure maximum performance and minimal client-side overhead. Aurora AI features a massive 3D Spatial UI with heavy Glassmorphism effects. By using Vanilla JS and a custom SPA (Single Page Application) router, we bypassed the virtual DOM overhead of React. This allowed for extremely smooth 60FPS animations, direct DOM manipulation for the tilt cards, and instant rendering of the 19-section Premium Cinematic Details Modal.

Q6: How does the application track user preferences in real-time?

Whenever a user interacts with a movie (e.g., clicking to open the details modal), the frontend

Aurora AI - Interview FAQs

asynchronously fires a POST request to '/events/ingest'. The Orchestrator Agent proxies this to the Event Processor microservice. The Event Processor updates the user's profile in an in-memory Feature Store. When the user asks for new recommendations, the RAG and Ranking engines query this Feature Store to bias the results towards the user's newly discovered interests.

Q7: Can you explain the Glassmorphism implementation?

Glassmorphism is achieved using CSS 'backdrop-filter: blur()' paired with semi-transparent rgba() background colors. For the cinematic modal, we used 'backdrop-filter: blur(40px) brightness(0.8)' combined with an inset border ('rgba(255, 255, 255, 0.1)') to create the frosted glass rim effect. We also utilized overlapping gradients to smoothly blend the movie's backdrop image into the dark UI.

Q8: What happens if one of the microservices fails?

The system is designed with graceful degradation. The Orchestrator Agent acts as the central router and wraps cross-service calls in try/catch blocks with strict timeouts. If the RAG service fails, the agent falls back to serving standard metadata. If the Ranking service fails, the system defaults to serving popular/trending movies from the static dataset, ensuring the user always receives a response rather than a 500 error.